EN3150 – Pattern Recognition

# Resource-Constrained CNN for Edge Image Classification

Assignment 03

Group Name: **RecogniX**

Ranmuthu S.G. - 230532M
Perera K.S.D. - 230480B
Suraweera B.U.D. - 230626F
Gunawardena H.A. - 230219K

**Abstract**

This report designs and evaluates small CNNs for head-pose classification (left, right, straight, up) on the CMU Face Images dataset. We compare a standard CNN (Model A) with a depth-wise separable CNN under 100,000 parameters (Model B), study Adam, SGD and SGD with momentum, and benchmark both against fine-tuned MobileNetV2 and EfficientNet-B0.

Code repository:
https://github.com/SandulRanmuthu/RocogniX_Classification_Assignment

# Contents

**1 Introduction** **2**

**2 Data Preparation** **2**
2.1 Dataset and Split . . . . . 2
2.2 Preprocessing . . . . . 2

**3 Custom Architecture Design** **3**
3.1 Model Specifications . . . . . 3
3.2 Parameter and MAC Calculations . . . . . 4
3.3 Hardware-Aware Activation Function . . . . . 4

**4 Optimizer Selection and Comparison** **4**

**5 Custom Model Training and Evaluation** **6**
5.1 Training Curves . . . . . 6
5.2 Test Set Evaluation . . . . . 7
5.3 Model A vs Model B . . . . . 8

**6 Comparison with Lightweight SOTA Networks** **8**
6.1 Selected Models and Fine-Tuning . . . . . 8
6.2 Overall Benchmark . . . . . 9

**7 Final Comparison and Trade-off Discussion** **10**

**8 Conclusion** **10**

# 1 Introduction

Edge devices (microcontrollers, Raspberry Pi, phones) have limited memory, compute and power, so models must be small and cheap per inference. This project:

1. builds a standard CNN (Model A) and a depthwise separable CNN with fewer than 100k parameters (Model B);
2. justifies the activation function and compares optimizers;
3. benchmarks the custom models against fine-tuned MobileNetV2 and EfficientNet-B0.

# 2 Data Preparation

## 2.1 Dataset and Split

The CMU Face Images dataset [3] has 640 grayscale face images with varying pose, expression, eyes (sunglasses or not) and size. We classify the pose (straight, left, right, up). The 64×60 images already meet the 64×64 limit. Corrupted `.bad` files were skipped, leaving 624 images, split randomly 70/15/15 (so the same subjects appear in all splits).

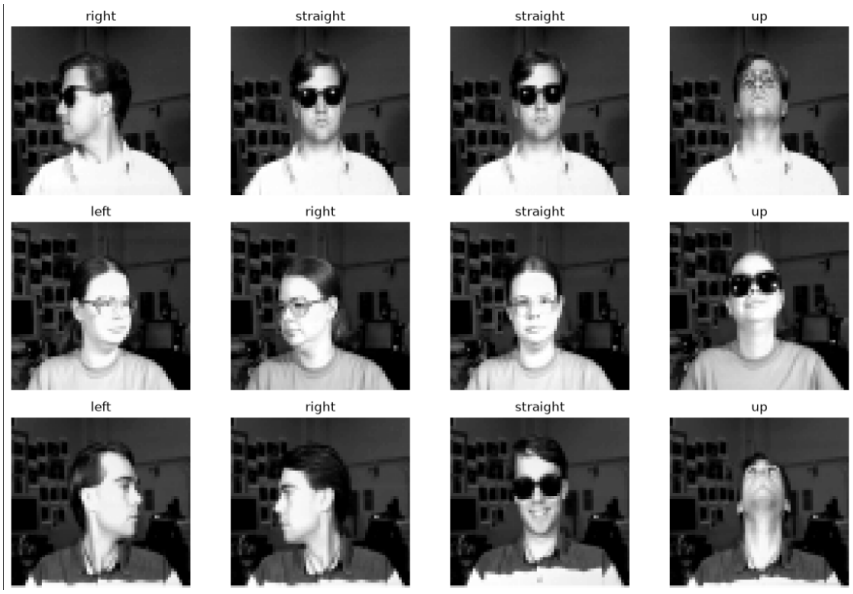



Figure 1: Representative 64×60 grayscale samples for each pose class.

Table 1: Class distribution across the data splits.

| Split | Left | Right | Straight | Up |
|---|---|---|---|---|
| Train (436) | 105 | 113 | 109 | 109 |
| Validation (93) | 25 | 23 | 26 | 19 |
| Test (95) | 27 | 19 | 21 | 28 |

## 2.2 Preprocessing

- Pixels were scaled to $[0, 1]$. For Models A and B they were also standardised with the training-set mean (0.2753) and std (0.2529).
- Custom models use one channel ($60 \times 64 \times 1$). For the pretrained models the channel is replicated to three, then rescaled inside the model ($[-1, 1]$ for MobileNetV2, $[0, 255]$ for EfficientNet-B0).

# 3 Custom Architecture Design

## 3.1 Model Specifications

- **Model A (Standard CNN):** four $3 \times 3$ convolutions (32, 64, 128, 128 filters), each followed by Batch Normalisation (BN, momentum 0.9) and ReLU, with max pooling after the first three. Global average pooling replaces flatten, then Dense(64), Dropout(0.3) and a 4-way softmax. Convolutions use He initialisation and no bias.
- **Model B (Lightweight CNN):** the same layout, with each convolution replaced by a depthwise $3 \times 3$ plus pointwise $1 \times 1$ convolution (192 filters in the last block, Dense(128) head). It has 62,925 trainable parameters, below the 100,000 limit.

Table 2: Layer-wise architecture of Model A.

| **Layer** | **Output Shape** | **Parameters** |
|---|---|---|
| Input | $60 \times 64 \times 1$ | 0 |
| Conv2D(32, 3×3) + BN + ReLU | $60 \times 64 \times 32$ | 288 + 128 |
| MaxPool 2×2 | $30 \times 32 \times 32$ | 0 |
| Conv2D(64, 3×3) + BN + ReLU | $30 \times 32 \times 64$ | 18,432 + 256 |
| MaxPool 2×2 | $15 \times 16 \times 64$ | 0 |
| Conv2D(128, 3×3) + BN + ReLU | $15 \times 16 \times 128$ | 73,728 + 512 |
| MaxPool 2×2 | $7 \times 8 \times 128$ | 0 |
| Conv2D(128, 3×3) + BN + ReLU | $7 \times 8 \times 128$ | 147,456 + 512 |
| GlobalAveragePooling2D | 128 | 0 |
| Dense(64) + ReLU + Dropout(0.3) | 64 | 8,256 |
| Dense(4) + Softmax | 4 | 260 |
| **Total** | | **249,828** |
| **Trainable** | | **249,124** |

Table 3: Layer-wise architecture of Model B.

| **Layer** | **Output Shape** | **Parameters** |
|---|---|---|
| Input | $60 \times 64 \times 1$ | 0 |
| SeparableConv2D(32, 3×3) + BN + ReLU | $60 \times 64 \times 32$ | 41 + 128 |
| MaxPool 2×2 | $30 \times 32 \times 32$ | 0 |
| SeparableConv2D(64, 3×3) + BN + ReLU | $30 \times 32 \times 64$ | 2,336 + 256 |
| MaxPool 2×2 | $15 \times 16 \times 64$ | 0 |
| SeparableConv2D(128, 3×3) + BN + ReLU | $15 \times 16 \times 128$ | 8,768 + 512 |
| MaxPool 2×2 | $7 \times 8 \times 128$ | 0 |
| SeparableConv2D(192, 3×3) + BN + ReLU | $7 \times 8 \times 192$ | 25,728 + 768 |
| GlobalAveragePooling2D | 192 | 0 |
| Dense(128) + ReLU + Dropout(0.3) | 128 | 24,704 |
| Dense(4) + Softmax | 4 | 516 |
| **Total** | | **63,757** |
| **Trainable** | | **62,925** |

### 3.2 Parameter and MAC Calculations

For kernel size $k$, $C_{in}$ input and $C_{out}$ output channels (no bias, since BN supplies the shift):

$$P_{\text{conv}} = k^2 C_{in} C_{out}, \quad \text{MACs}_{\text{conv}} = HWk^2 C_{in} C_{out} \tag{1}$$

$$P_{\text{sep}} = k^2 C_{in} + C_{in} C_{out}, \quad \text{MACs}_{\text{sep}} = HWk^2 C_{in} + HWC_{in} C_{out} \tag{2}$$

For the second layer ($C_{in} = 32$, $C_{out} = 64$, output $30 \times 32$): Model A needs 18,432 parameters and 17,694,720 MACs; Model B needs 2,336 parameters and 2,242,560 MACs (7.9× fewer). In total, Model A needs **44.76 M MACs** per image and Model B **5.97 M** (7.5× fewer).

### 3.3 Hardware-Aware Activation Function

We use **ReLU** ($\max(0, x)$) in all hidden layers. It needs only a comparison, with no exponentials, so it is cheap on CPUs and microcontrollers without an FPU, and it is quantisation-friendly (int8). Sigmoid, tanh and swish all need exponentials. Softmax is used only once, at the output.

## 4 Optimizer Selection and Comparison

We selected **Adam** ($10^{-3}$): its per-parameter adaptive step sizes suit layers with very different gradient scales and need little tuning. It was compared with plain SGD ($5 \times 10^{-2}$) and SGD with momentum (0.9, $10^{-2}$) on the same model, initial weights and data. Model A was trained for 100 epochs with `ReduceLROnPlateau` (learning rate halved after 5 epochs without validation-loss improvement); Model B for 50 epochs with a constant learning rate.

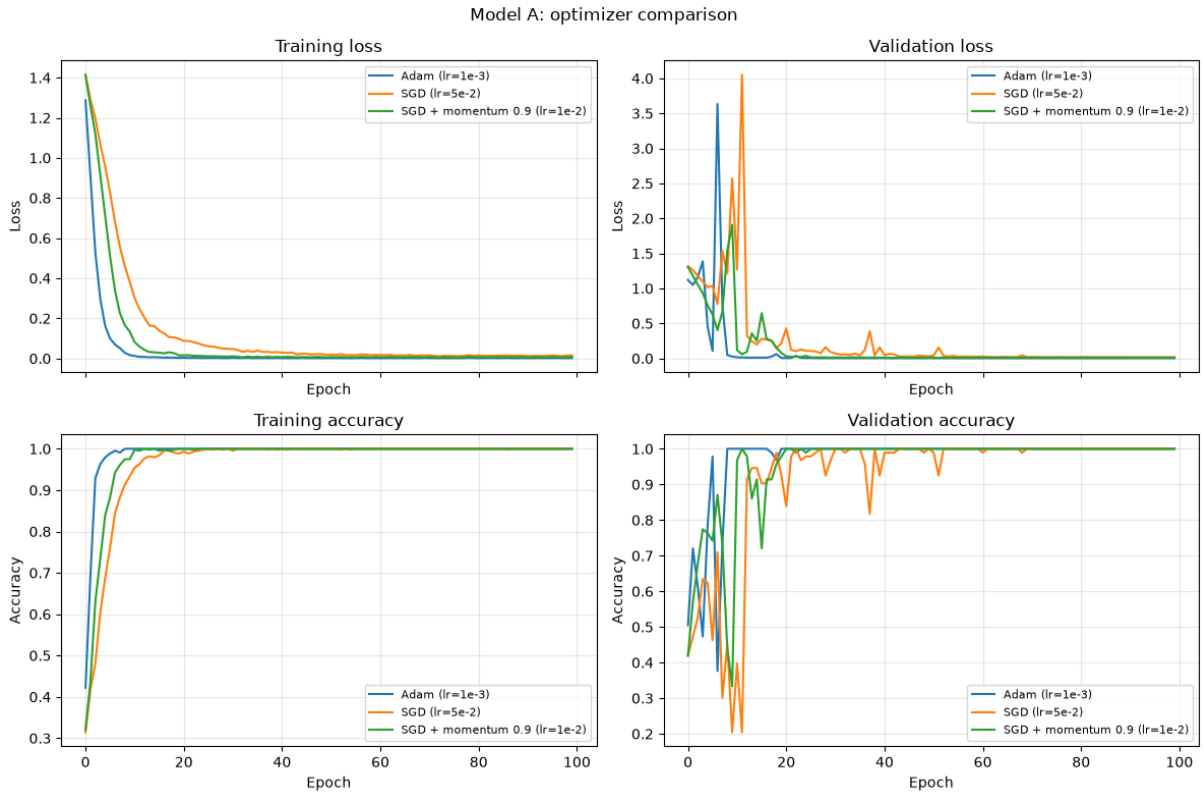



Figure 2: Loss and accuracy curves for the three optimizers of Model A.

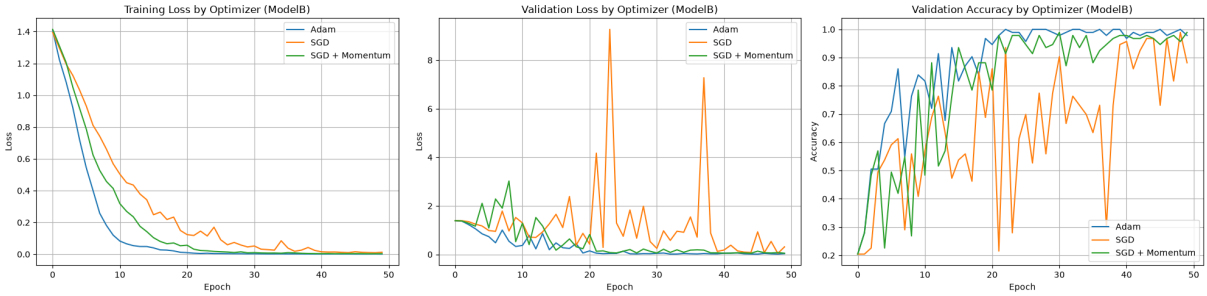



Figure 3: Loss and accuracy curves for the three optimizers of Model B.

Table 4: Optimizer comparison. Epoch to 90% is the first epoch with validation accuracy $\geq$ 90% (Model A).

| **Optimizer** | **Model A: Epoch to 90%** | **Model A Test Acc** | **Model B Test Acc** |
|---|---|---|---|
| Adam ($10^{-3}$) | 6 | 0.989 | 0.947 |
| SGD ($5 \times 10^{-2}$) | 13 | 0.968 | 0.737 |
| SGD + Momentum 0.9 ($10^{-2}$) | 11 | 0.979 | 0.947 |

- **Adam** converged fastest and gave the best test accuracy on both models.
- **Plain SGD** is the noisiest. On Model B, without learning-rate decay, its validation loss spiked repeatedly (up to about 9) and it ended at 73.7% test accuracy. Without input standardisation and BN,on an version of Model A SGD never left the chance-level plateau (validation accuracy 0.247).
- **SGD with momentum** matched Adam's final accuracy on Model B and was close on Model A.

**Effect of momentum.** Momentum accumulates past gradients, so the effective step is about $\eta/(1-\beta)$. On Model A, raising $\beta$ from 0 to 0.9 cut the epochs to reach 90% validation accuracy from 33 to 11 (Table 5). $\beta = 0.99$ (effective step 1.0) overshot: its validation loss spiked to about 15 and validation accuracy kept dropping until around epoch 30. All settings eventually reached 100% validation accuracy, so momentum mainly affects convergence speed and stability.

Table 5: Momentum sweep on Model A (SGD, $\eta = 10^{-2}$).

| **Momentum $\beta$** | **Effective step** | **Best Val Acc** | **Epoch to 90% Val** |
|---|---|---|---|
| 0.0 | 0.01 | 1.000 | 33 |
| 0.5 | 0.02 | 1.000 | 17 |
| 0.9 | 0.10 | 1.000 | 11 |
| 0.99 | 1.00 | 1.000 | 11 |

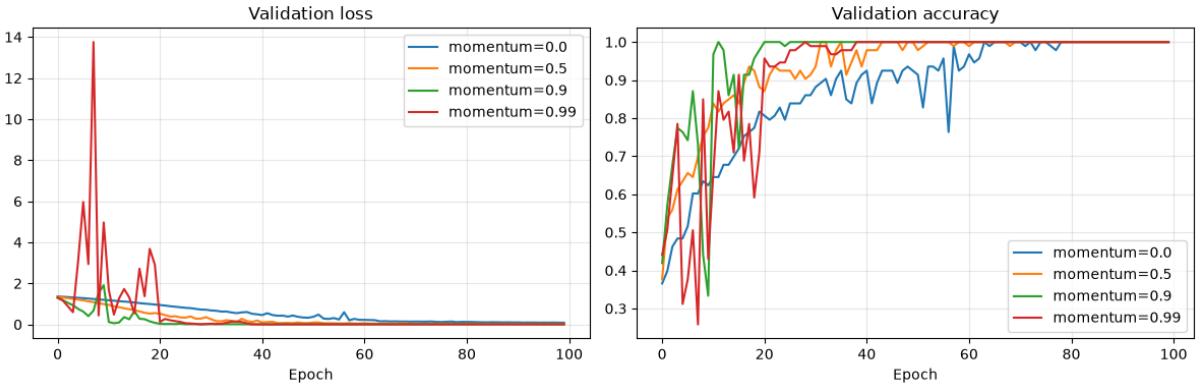


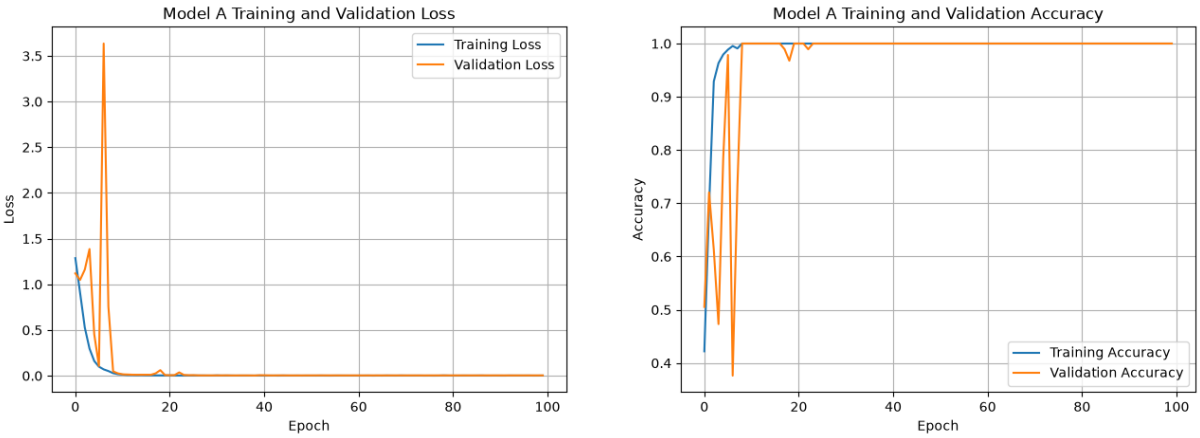

Figure 4: Effect of the momentum parameter on validation loss and accuracy (Model A).

# 5 Custom Model Training and Evaluation

## 5.1 Training Curves

Both models use Adam ($10^{-3}$), batch size 32 and cross-entropy loss. Model A was trained for 100 epochs and Model B for 50.

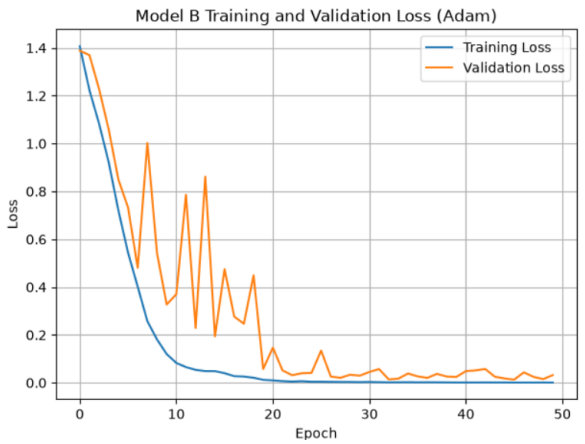
- **Model A:** validation loss has one spike (about 3.8 at epoch 7) while BN statistics adapt, then stays flat; validation accuracy reaches 100%.
- **Model B:** validation loss and accuracy fluctuate for the first ∼20 epochs, then settle at 98–100% (final 97.85%). The gap between training loss (0.0008) and validation loss (0.032) indicates mild overfitting; we suspect this is due to the small number of training/validation samples.



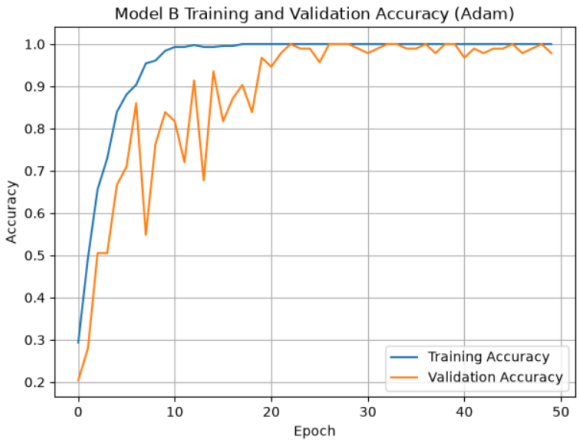

(a) Model A loss

(b) Model A accuracy

Figure 5: Training and validation curves of Model A.



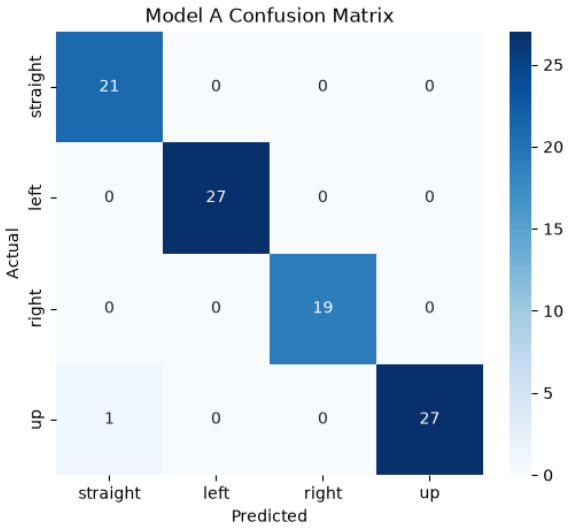

(a) Model B loss



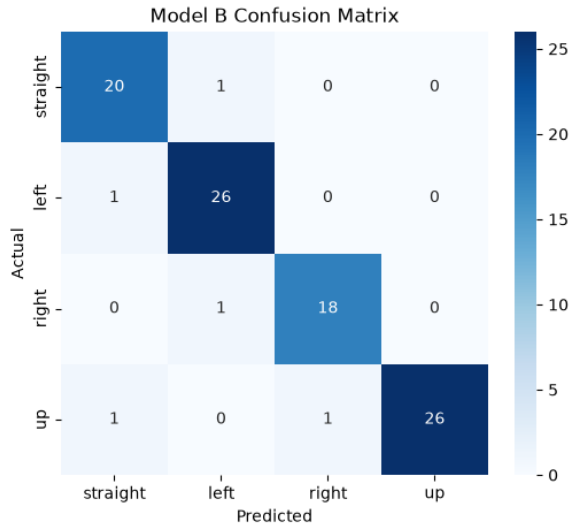

(b) Model B accuracy

Figure 6: Training and validation curves of model B.

## 5.2 Test Set Evaluation




(a) Model A




(b) Model B

Figure 7: Confusion matrices on the test set.

Model A makes 1 error in 95 images (one up predicted as straight). Model B makes 5 errors, spread over straight, left and right, with two up images missed. Straight is the least distinctive pose, since a slightly turned face looks similar to a straight one at 64×60 resolution.

### 5.3 Model A vs Model B

Table 6: Comparison of the custom models.

| Metric | Model A (Standard) | Model B (Separable) |
|---|---|---|
| Trainable parameters | 249,124 | 62,925 |
| Total parameters (incl. BN statistics) | 249,828 | 63,757 |
| MACs | 44.76 M | 5.97 M |
| FP32 weight size (KB) | 975.9 | 249.1 |
| Model size on disk (KB) | 1040.3 | 315.9 |
| Training time per epoch (s) | 1.135 | 0.977 |
| Epochs trained | 100 | 50 |
| Test accuracy (%) | 98.95 | 94.74 |
| Test precision / recall (macro) | 0.9886 / 0.9911 | 0.9463 / 0.9478 |
| Test F1 (macro) | 0.9896 | 0.9465 |

**Trade-off.** Depthwise separable convolutions cut parameters by about 4× and MACs by about 7.5×, for a loss of about 4 points of accuracy (4 more errors on 95 images). Training time per epoch is similar; the difference is within timing noise, because depthwise convolutions are memory-bound and less optimised on desktop CPUs. The MAC saving pays off on edge runtimes with optimised kernels.

## 6 Comparison with Lightweight SOTA Networks

### 6.1 Selected Models and Fine-Tuning

- **MobileNetV2** [1]: inverted residual blocks with linear bottlenecks.
- **EfficientNet-B0** [2]: compound scaling, squeeze-and-excitation and swish activations.

Both use ImageNet weights, the same data splits and a new head (global average pooling, Dense(128), Dropout(0.3), Dense(4)). Training has two stages of 20 epochs: (1) frozen backbone, Adam $10^{-3}$; (2) last 30 backbone layers unfrozen (BN frozen), Adam $10^{-5}$.

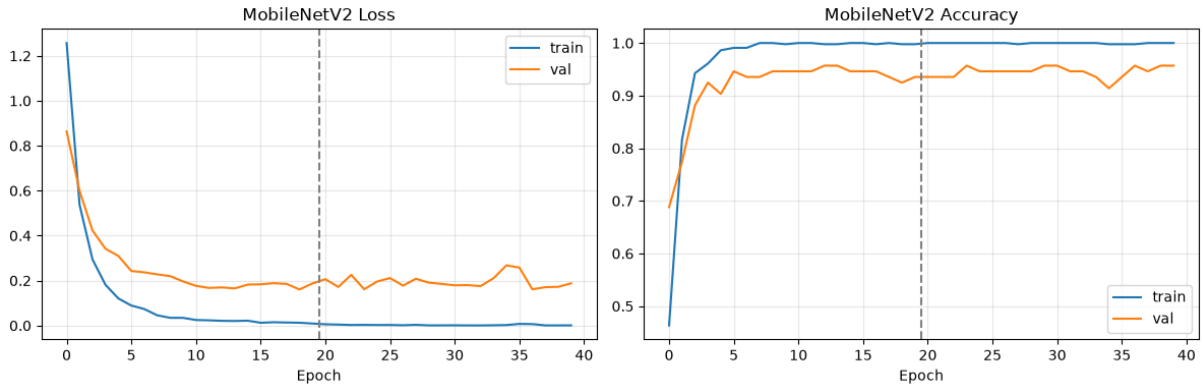



Figure 8: Training and validation curves during head training and fine-tuning of MobileNetV2 model.

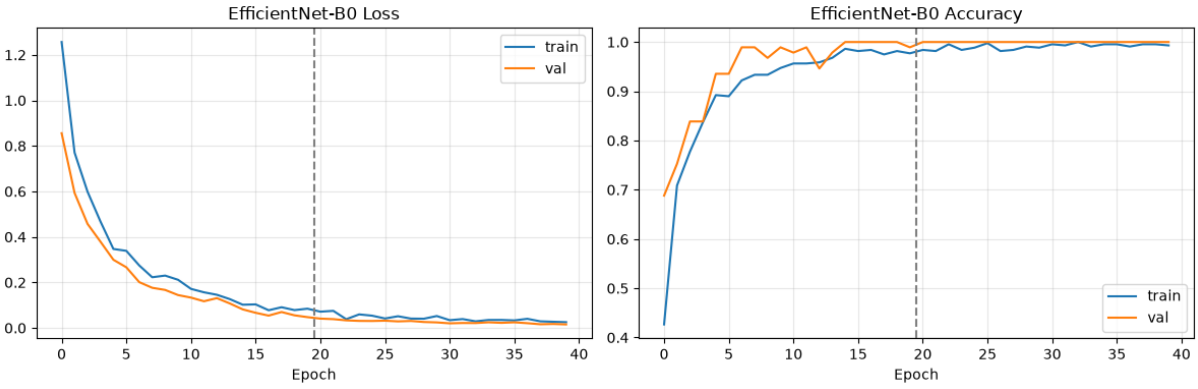
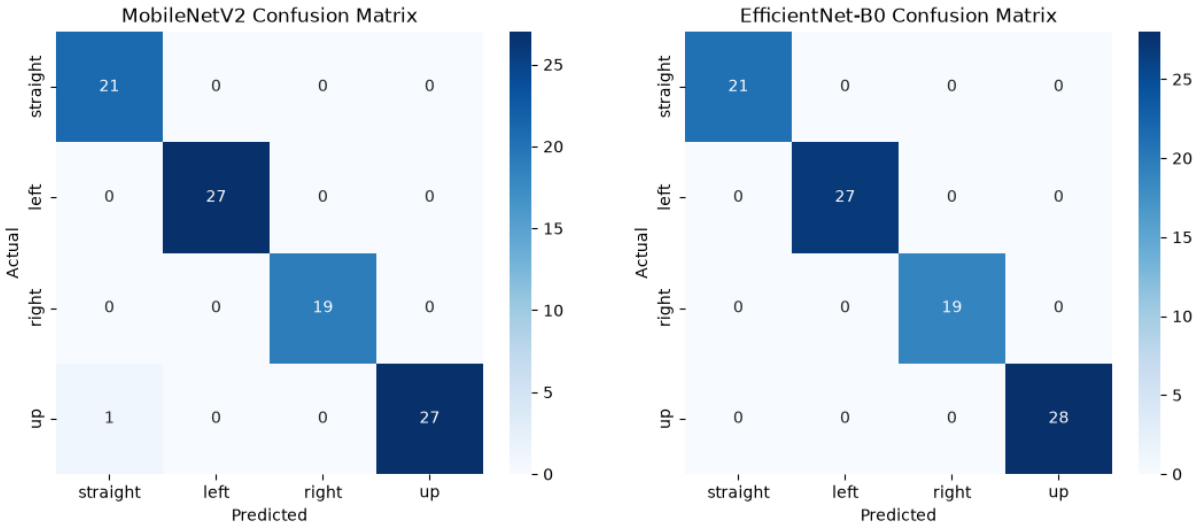



Figure 9: Training and validation curves during head training and fine-tuning of EfficientNet-B0 model.

EfficientNet-B0 reached 100% validation accuracy after 15 epochs of head training and stayed there during fine-tuning, with no sign of overfitting. MobileNetV2 reached about 94–96% in stage 1; in stage 2 its validation loss remained noisy while the training loss approached zero, a sign of overfitting.




(a) MobileNetV2

(b) EfficientNet-B0

Figure 10: Confusion matrices of the fine-tuned models on the test set.

## 6.2 Overall Benchmark

Table 7: Comparison of custom and fine-tuned models. Size is FP32 weight size; MACs are per $60 \times 64$ image.

| Model | Total Params | Size (MB) | MACs | s/epoch | Test Acc (%) | Macro F1 |
|---|---|---|---|---|---|---|
| Model A | 249,828 | 0.95 | 44.8 M | 1.135 | 98.95 | 0.9896 |
| Model B | 63,757 | 0.24 | 5.97 M | 0.977 | 94.74 | 0.9465 |
| MobileNetV2 | 2,422,468 | 9.24 | 24.2 M | 0.743 / 1.076* | 98.95 | 0.9896 |
| EfficientNet-B0 | 4,214,055 | 16.08 | 31.7 M | 1.109 / 1.590* | 100.00 | 1.0000 |

* Head training / fine-tuning stage.

# 7 Final Comparison and Trade-off Discussion

- **Accuracy vs memory:** EfficientNet-B0 is the most accurate (100%) but needs about 16 MB. Model A matches MobileNetV2 (98.95%) with about 10× fewer parameters. Model B is 38× smaller than MobileNetV2 and 66× smaller than EfficientNet-B0, for a loss of 4–5 points of accuracy.
- **Computational cost:** parameters and MACs do not go together. Model A has few parameters but needs more MACs (44.8 M) than MobileNetV2 (24.2 M) or EfficientNet-B0 (31.7 M). Model B is cheapest on both counts.
- **Edge targets:** Model B is suitable for microcontrollers (about 250 KB in FP32, less when quantised); Model A or MobileNetV2 for a Raspberry Pi; MobileNetV2 or EfficientNet-B0 for phones.
- **Scratch vs transfer learning:** a custom sub-100k model gives full control over size and latency but needs careful design and gave lower accuracy. Pretrained models converge quickly and generalise better from small data, but are larger and fixed in design.
- **Limitations:** the dataset is small (95 test images, so one image is about 1 point) and subjects are shared across splits. So model accuracy cannot be guaranteed and may also have cause overfitting in some cases.

# 8 Conclusion

We built a standard CNN (98.95% test accuracy) and a depthwise separable CNN under 100k parameters (94.74%) for head-pose classification. Adam converged most reliably, and momentum 0.9 sped up SGD about three times. Model B uses about 4× fewer parameters and 7.5× fewer MACs than Model A. Fine-tuned MobileNetV2 and EfficientNet-B0 reached 98.95% and 100% but are 38–66× larger than Model B. For strongly resource-constrained edge deployment we recommend **Model B**.

# References


[1] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proc. CVPR*, 2018.

[2] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proc. ICML*, 2019.

[3] T. Mitchell, "CMU Face Images," UCI Machine Learning Repository, 1997.